

CSA0613-Design and Analysis of Algorithms

NAME: Safiya B

REG NO: 192411043

Case Study: Employee Database Search (Binary Search)

Aim

To implement Binary Search for searching an employee record from a sorted employee database efficiently.

Problem Statement

A corporate company stores employee records in sorted order based on Employee ID. Implement Binary Search to quickly find an employee record and analyze its efficiency.

Requirements

- Employee records must be sorted by Employee ID.
- Fast random access to records.
- Use the Divide and Conquer technique.
- Display whether the employee is found or not.
- Analyze the time complexity.

Algorithm

1. Read the number of employee IDs.
2. Store employee IDs in sorted order.
3. Read the Employee ID to search.
4. Set $low = 0$ and $high = n-1$.
5. Find $mid = (low + high)/2$.
6. If the key equals the middle element, display "Employee Found".
7. If the key is smaller, search the left half.
8. Otherwise, search the right half.

9. Repeat until found or $low > high$.

Source Code

```
def binary_search(a, key):  
    low, high = 0, len(a) - 1  
  
    while low <= high:  
        mid = (low + high) // 2  
  
        if a[mid] == key:  
            return mid  
        elif a[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    return -1  
  
emp = [101, 105, 110, 115, 120, 125, 130, 135]  
  
key = int(input("Enter Employee ID: "))  
  
result = binary_search(emp, key)  
  
if result != -1:  
    print("Employee Found at position", result + 1)  
else:  
    print("Employee Not Found")
```

Output

```
Enter Employee ID: 125
Employee Found at position 6

==<== Code Execution Successful ===|
```

Divide and Conquer

- Binary Search divides the sorted list into two halves.
- It compares the search key with the middle element.
- If the key is smaller, it searches the left half.
- If larger, it searches the right half.
- This process continues until the element is found or the search space becomes empty.

Time Complexity

Case	Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Space Complexity: $O(1)$

Advantages

- Very fast for large sorted datasets.
- Reduces the number of comparisons.
- Efficient for employee database lookup.
- Uses the Divide and Conquer strategy.